

UNIVERSITÉ PARIS DAUPHINE – PSL

LAMSADE

Master – Module : **Qualité de Données, Data Wrangling**

Encadrant : **Khalid Belhajjame**

Découverte de Pattern Functional Dependencies

Approche Classique vs. Agentique (IA)

SOLIMAN Omar **DERRADJI** Fahd

HADDAR Djena **TSOULI** Abderrahmane

KARAOUANE Guillaume

Année universitaire 2025 – 2026

Table des matières

1	Introduction	3
1.1	Contexte et motivations	3
1.2	Objectifs du projet	3
1.3	Organisation du rapport	3
1.4	Conformité au cahier des charges du cours	3
2	Concepts théoriques	4
2.1	Dépendances fonctionnelles classiques	4
2.2	Pattern Functional Dependencies (PFDs)	5
2.3	PFDs approximatives : support et confiance	5
2.4	Transformations utilisées	6
3	Architecture du système	6
3.1	Vue d'ensemble	6
3.2	Technologies utilisées	7
3.3	Datasets	7
4	Approche classique	7
4.1	Pipeline en 5 étapes	7
4.1.1	Étape 1 : Extraction de patterns	8
4.1.2	Étape 2-3 : Génération de candidats	8
4.1.3	Étape 4 : Validation	8
4.1.4	Étape 5 : Généralisation	9
5	Approche agentique	9
5.1	Principe général	9
5.2	Intégration LLM via Ollama	9
5.3	Workflow 1 : Feature-Enriched Discovery	9
5.4	Workflow 2 : Guided Search	10
5.5	Workflow 3 : Agent-in-the-Loop	10
5.6	Généralisation sémantique par LLM	11
5.7	Comparaison des workflows	12
6	Résultats expérimentaux	12
6.1	Paramètres expérimentaux	12
6.2	Résultats de l'approche classique	12
6.2.1	Exemples de PFDs découvertes	13
6.3	Résultats de l'approche agentique	13
6.3.1	Comparaison globale sur t1.csv	13
6.3.2	Analyse de la réduction de l'espace de recherche	14
6.3.3	Suggestions des agents	14
6.3.4	Candidats prioritaires (Workflow 2)	15
6.4	Comparaison Mistral vs. Llama	15
6.5	Résultats du Workflow 3 (Agent-in-the-Loop)	15
6.6	Analyse de sensibilité aux seuils K et θ	17
6.7	Généralisation sémantique par LLM vs. algorithmique	18

6.8	Analyse d'erreur : pourquoi Mistral W2 manquait des PFDs parfaites . . .	18
7	Discussion	19
7.1	Avantages de l'approche agentique	19
7.2	Limites et compromis	20
7.3	Analyse du compromis qualité/exhaustivité	20
7.4	Reproductibilité	20
8	Conclusion	21
A	Annexe : Guide d'utilisation	23
A.1	Installation	23
A.2	Exécution	23
B	Annexe : Structure du code	23

1 Introduction

1.1 Contexte et motivations

La qualité des données est un enjeu majeur dans les systèmes d'information modernes. Les **dépendances fonctionnelles** (FDs) constituent un outil fondamental pour détecter les anomalies, nettoyer les données et comprendre la structure sémantique d'un jeu de données. Cependant, les FDs classiques comparent des **valeurs entières** et ne capturent pas les régularités au niveau des *patterns* (préfixes, tokens, sous-chaînes).

Les **Pattern Functional Dependencies** (PFDs) généralisent les FDs en utilisant des transformations sur les valeurs. Par exemple, la règle `prefix(zip, 3) → city` exprime que les 3 premiers chiffres d'un code postal déterminent généralement la ville — une régularité invisible aux FDs classiques.

1.2 Objectifs du projet

Ce projet poursuit plusieurs objectifs :

1. **Implémenter un algorithme classique** de découverte de PFDs, suivant un pipeline en 5 étapes : extraction de patterns, groupement, génération de candidats, validation et généralisation.
2. **Concevoir et implémenter une approche agentique** utilisant des LLMs (Large Language Models) locaux pour guider intelligemment la recherche de PFDs.
3. **Implémenter les TROIS workflows agentiques** présentés dans le cours : Feature-Enriched (W1), Guided Search (W2) et Agent-in-the-Loop (W3, ajouté dans la version 2 du projet).
4. Compléter le pipeline classique par une **généralisation sémantique par LLM** (étape 5 LLM-aware), une **analyse de sensibilité** aux seuils K et θ , et une **analyse d'erreur détaillée** des PFDs manquées par les workflows agentiques.

Nous comparons deux LLMs locaux — **Mistral 7B** et **Llama 3.1 8B** — exécutés via Ollama, sur 14 datasets réels couvrant trois domaines : gouvernance de données, chimie/biologie moléculaire et validation.

1.3 Organisation du rapport

La section 2 présente les concepts théoriques fondamentaux. La section 3 décrit l'architecture du système. La section 4 détaille l'approche classique. La section 5 présente l'approche agentique. La section 6 analyse les résultats expérimentaux. La section 7 discute les enseignements et limites. La section 8 conclut le rapport.

1.4 Conformité au cahier des charges du cours

Le tableau 1 détaille la correspondance entre les exigences du cours (*Approximate_PFDs*) et les livrables de ce projet.

TABLE 1 – Couverture des exigences du cours

Exigence	Statut	Référence dans le rapport
Tâche 1 – Pipeline classique (extraction, candidats, validation)	✓	Section 4
Tâche 2 – Au moins un workflow agentique	✓	3 workflows implémentés (sections 5.3, 5.4, 5.5)
Tâche 3 – Expériences quantitatives et qualitatives	✓	Section 6 (14 datasets)
Définition formelle PFD $R(X \rightarrow Y, T_p)$	✓	Section 2.2
PFDs approximatives (support, confidence)	✓	Section 2.3
Pipeline en 5 étapes	✓	Sections 4.1.1 à 4.1.5
Forme pattern (X) $\rightarrow$ Y (vs. bilatérale)	✓	Section 2.2, paragraphe dédié
Métriques : support, confidence, performance	✓	Sections 6.2, 6.3
Nombre de candidats explorés	✓	Tableau 12
Trade-off qualité vs. coût computationnel	✓	Section 7.3 (tableau 21)
Comparaison de ≥ 2 LLMs	✓	Mistral 7B vs. Llama 3.1 8B (section 6.4)
Description de l'approche	✓	Sections 4 et 5
Résultats expérimentaux	✓	Section 6
Comparaison entre méthodes	✓	Tableau 11 et 13
Discussion et insights	✓	Section 7

Dépassement du cahier des charges. Le projet dépasse les exigences minimales sur plusieurs points :

- Les **trois** workflows agentiques sont implémentés (un seul était requis).
- Une **analyse de sensibilité** aux seuils K et θ a été menée (section 6.6).
- Une **généralisation sémantique par LLM** complète la généralisation algorithmique (section 5.6).
- Une **analyse d'erreur reproductible** diagnostique les PFDs manquées par les LLMs (section 6.8).

2 Concepts théoriques

2.1 Dépendances fonctionnelles classiques

Définition – Dépendance Fonctionnelle

Soit une relation R sur un schéma $\mathcal{S}$. Une **dépendance fonctionnelle** $X \rightarrow Y$ (où $X, Y \subseteq \mathcal{S}$) tient si et seulement si :

$$\forall t_1, t_2 \in R : t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$$

En d'autres termes, si deux tuples ont la même valeur sur X , ils doivent avoir la même valeur sur Y . Par exemple, dans une table d'employés, **Department** $\rightarrow$ **Department Name** signifie que le code du département détermine de manière unique son nom complet.

Limites des FDs classiques :

- Elles comparent des **valeurs entières** uniquement.
- Elles ne détectent pas que les prénoms commençant par « John » sont masculins.

- Elles ne capturent pas que les codes postaux avec le même préfixe de 3 chiffres correspondent à la même ville.

2.2 Pattern Functional Dependencies (PFDs)

Définition – Pattern Functional Dependency

Une PFD $R(X \rightarrow Y, T_p)$ généralise les FDs en utilisant un **tableau de patterns** T_p . Elle tient si :

$$\forall t_1, t_2 \in R : T_p(t_1[X]) = T_p(t_2[X]) \Rightarrow t_1[Y] = t_2[Y]$$

où T_p est une transformation appliquée aux valeurs de X .

Exemples concrets :

- `first_token(name) → gender` : le prénom détermine le genre
- `prefix(zip, 3) → city` : les 3 premiers chiffres du code postal déterminent la ville
- `first_token(ref_id) → ref_type` : le premier token d'un identifiant de référence détermine son type (PubMed, DailyMed, etc.)

Choix de la forme $\text{pattern}(X) \rightarrow Y$. Le cadre général des PFDs autorise la forme bilatérale $\text{pattern}(X) \rightarrow \text{pattern}(Y)$. Nous nous restreignons à la forme unilatérale $\text{pattern}(X) \rightarrow Y$ pour deux raisons : (1) c'est la forme la plus utile en pratique pour la qualité de données (data cleaning), comme indiqué dans le cours, et (2) elle est plus interprétable pour un humain. Notre T_p porte donc uniquement sur le côté X .

2.3 PFDs approximatives : support et confiance

Les données réelles contiennent du **bruit**. Une PFD ne tient donc pas à 100%. On utilise deux métriques pour quantifier la qualité d'une PFD approximative :

Métriques de qualité

Support : nombre de tuples couverts par les groupes formés par le pattern X :

$$\text{support}(X) = \sum_{g \in \text{Groups}(X)} |g|$$

Confiance : proportion de tuples « cohérents » (qui respectent la dépendance) :

$$\text{conf}(X \rightarrow Y) = \frac{\sum_{g \in \text{Groups}(X)} \text{count}(\text{majority}_Y(g))}{\sum_{g \in \text{Groups}(X)} |g|}$$

Bruit (Noise) : taux de violation, complémentaire de la confiance :

$$\text{noise} = 1 - \text{conf}(X \rightarrow Y)$$

On ne retient une PFD que si elle satisfait deux seuils :

- $\text{support} \geq K$ (dans nos expériences, $K = 5$)

— confidence $\geq \theta$ (dans nos expériences, $\theta = 0.85$)

2.4 Transformations utilisées

Notre système supporte 7 types de transformations, résumés dans le tableau 2.

TABLE 2 – Les 7 transformations supportées par le système

Transformation	Notation	Exemple	Usage typique
Identity	<code>identity(col)</code>	« Chicago » → « Chicago »	Catégoriques
Prefix	<code>prefix(col, k)</code>	« 90012 » → « 900 »	Codes postaux
Suffix	<code>suffix(col, k)</code>	« report.pdf » → « pdf »	Extensions
First token	<code>first_token(col)</code>	« John Smith » → « John »	Prénoms
Last token	<code>last_token(col)</code>	« John Smith » → « Smith »	Noms de famille
Numeric prefix	<code>numeric_prefix(col, k)</code>	« ZIP-90012 » → « 900 »	Données bruitées
Length	<code>length(col)</code>	« Hello » → « 5 »	Longueur

3 Architecture du système

3.1 Vue d'ensemble

Le projet est organisé en modules Python indépendants et réutilisables, répartis sur 17 fichiers. L'architecture suit une séparation nette entre le cœur algorithmique, l'intégration LLM et les scripts d'expérimentation.

TABLE 3 – Organisation des modules du projet

Module	Rôle
<i>Cœur algorithmique et intégration LLM (src/)</i>	
<code>data_loader.py</code>	Chargement CSV, introspection du schéma
<code>pattern_extraction.py</code>	Définition et application des 7 transformations
<code>pattern_grouping.py</code>	Groupement des tuples par pattern
<code>candidate_generation.py</code>	Génération des paires candidats (T_X, Y)
<code>validation.py</code>	Calcul du support et de la confiance
<code>generalization.py</code>	Fusion algorithmique + sémantique (LLM)
<code>classical_pipeline.py</code>	Orchestration du pipeline en 5 étapes
<code>llm_agent.py</code>	Mistral/Llama + 5 prompts (W1, W2, W3, gen. sém.)
<code>agentic_workflow.py</code>	Workflows agentiques 1, 2 et 3
<code>evaluation.py</code>	Métriques et comparaison
<i>Scripts d'expérimentation (experiments/)</i>	
<code>run_classical.py</code>	Script classique
<code>run_agentic.py</code>	Script W1 et W2
<code>run_workflow3.py</code>	Script W3 (Agent-in-the-Loop)
<code>compare_results.py</code>	Comparaison classique vs W1/W2
<code>sensitivity_analysis.py</code>	Analyse de sensibilité K et θ
<code>error_analysis.py</code>	Analyse d'erreur W2
<code>run_llm_generalization.py</code>	Généralisation sémantique par LLM

3.2 Technologies utilisées

TABLE 4 – Stack technologique

Technologie	Usage
Python 3.10+	Langage principal
pandas	Manipulation de données tabulaires
Ollama	Serveur de LLMs locaux (REST API sur <code>localhost:11434</code>)
Mistral 7B	LLM local – 7 milliards de paramètres (~ 4.4 Go)
Llama 3.1 8B	LLM local – 8 milliards de paramètres (~ 4.9 Go)

Le choix de LLMs locaux (via Ollama) nous permet de travailler **sans quota**, **sans clé API**, et **hors-ligne**. Les deux modèles tournent confortablement sur un MacBook Air M2 avec 16 Go de RAM.

3.3 Datasets

Nous utilisons **14 fichiers CSV** répartis en 3 catégories, représentant un total de **46 351 lignes** et **103 colonnes** (tableau 5).

TABLE 5 – Datasets utilisés

Catégorie	Fichier	Lignes	Cols	Domaine
pfd_validation/	US_Phone_Code.csv	51	3	Téléphone US
	t1.csv	9 101	9	Employés gouv.
	t2.csv	3 502	13	Entreprises
	t3.csv	1 077	9	Licences
CHE/	mechanism_refs.csv	9 536	5	Réf. pharma
	metabolism_refs.csv	2 409	5	Métabolisme
	protein_classification.csv	858	7	Protéines
	research_companies.csv	812	5	Entr. pharma
	variant_sequences.csv	1 200	7	Séquences
DGOV/	570-1.csv	9 101	9	Employés gouv.
	10492-1.csv	1 077	9	Licences
	10642-1.csv	920	6	Agences emploi
	6339-1.csv	306	7	Criminalité
	6397-1.csv	6 704	9	Démographie

4 Approche classique

4.1 Pipeline en 5 étapes

L’approche classique suit un pipeline séquentiel en 5 étapes, illustré par la figure 1.

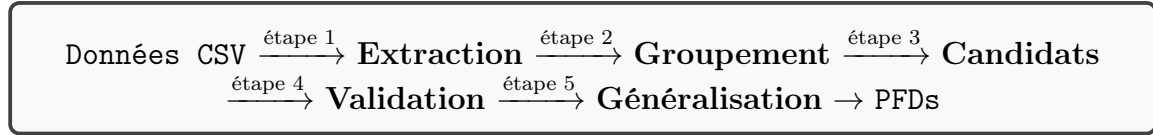


FIGURE 1 – Pipeline classique de découverte de PFDs

4.1.1 Étape 1 : Extraction de patterns

Pour chaque colonne du dataset, le système génère automatiquement les transformations pertinentes. La fonction `generate_default_transformations()` analyse chaque colonne et produit :

- `identity` pour toutes les colonnes.
- `first_token` et `last_token` si la colonne contient des espaces ou virgules (noms, adresses).
- `prefix(col, k)` pour $k \in [1, \min(\bar{L}, 5)]$ où $\bar{L}$ est la longueur moyenne des valeurs.
- `numeric_prefix(col, k)` si la colonne contient des chiffres.
- `length` pour toutes les colonnes.

Exemple : sur `t1.csv` (9 colonnes), cette étape génère environ **60 transformations**.

4.1.2 Étape 2–3 : Génération de candidats

Le système forme le produit cartésien des transformations et des colonnes cibles, en excluant les auto-dépendances triviales (`identity(X) → X`). Pour `t1.csv`, cela produit **~544 candidats**.

4.1.3 Étape 4 : Validation

L’algorithme de validation est le cœur du système. Pour chaque candidat $T(X) \rightarrow Y$:

Algorithm 1 Validation d’un candidat PFD

Require: DataFrame D , transformation T , colonne cible Y , seuils K, θ

Ensure: PFDResult ou $\perp$

```

1:  $groups \leftarrow \text{groupBy}(D, T)$  {Grouper par pattern}
2: Filtrer les groupes avec  $|g| < K$ 
3:  $total \leftarrow 0$ ,  $consistent \leftarrow 0$ 
4: for chaque groupe  $g$  dans  $groups$  do
5:    $counts \leftarrow \text{valueCounts}(g[Y])$ 
6:    $majority \leftarrow \max(counts)$ 
7:    $total \leftarrow total + |g|$ 
8:    $consistent \leftarrow consistent + majority$ 
9: end for
10:  $conf \leftarrow consistent/total$ 
11: if  $total \geq K$  and  $conf \geq \theta$  then
12:   return PFDResult( $T, Y, total, conf$ )
13: else
14:   return  $\perp$ 
15: end if
  
```

Complexité : $\mathcal{O}(C \times n \log n)$ où C est le nombre de candidats et n le nombre de lignes.

4.1.4 Étape 5 : Généralisation

La généralisation élimine les PFDs redondantes via trois stratégies :

1. **Préfixes** : parmi `prefix(col, 1)`, `prefix(col, 2)`, ..., garder le plus court avec une confiance $\geq$ (meilleure confiance -0.05).
2. **Subsomption** : si `identity(col)` a une bonne confiance, elle subsume les transformations sur la même colonne.
3. **Déduplication** : éliminer les doublons par type de transformation et colonne cible.

Exemple : sur `t1.csv`, la généralisation réduit **143 PFDs valides** à **85 PFDs généralisées**.

5 Approche agentique

5.1 Principe général

L'idée centrale est de combiner la **rigueur algorithmique** du pipeline classique avec le **raisonnement sémantique** d'un LLM. Le modèle de langage analyse les noms de colonnes et des échantillons de valeurs pour suggérer des transformations *pertinentes*, réduisant ainsi l'espace de recherche.

Nous implémentons deux workflows distincts, de complexité croissante.

5.2 Intégration LLM via Ollama

Les deux LLMs sont appelés via l'API REST d'Ollama (`POST /api/generate`). Chaque appel envoie un prompt structuré et reçoit une réponse en JSON. Les paramètres utilisés :

- `temperature` = 0.1 (réponses déterministes, peu de variabilité)
- `num_predict` = 4096 (longueur maximale de la réponse)
- `stream` = false (réponse complète en une fois)

5.3 Workflow 1 : Feature-Enriched Discovery

Workflow 1 – Feature-Enriched Discovery

Table $R \rightarrow$ **Agent LLM** $\rightarrow$ Transformations suggérées $\rightarrow$ **Algo classique** $\rightarrow$ PFDs

Fonctionnement détaillé :

1. **Analyse du schéma** : le système extrait pour chaque colonne le nombre de valeurs uniques et 5 échantillons de valeurs.
2. **Appel LLM** : un prompt structuré demande au LLM de suggérer des transformations pertinentes parmi les 7 types disponibles, avec justification.

3. **Parsing** : les suggestions JSON sont converties en objets **Transformation**.
4. **Pipeline classique** : les étapes 2 à 5 du pipeline classique sont exécutées avec les transformations suggérées par l'agent.

Exemple de suggestion (Mistral sur t1.csv) :

```

1 {
2   "name": "first_token",
3   "column": "Full Name",
4   "params": {},
5   "reason": "Le premier mot du nom complet est le prenom"
6 }
```

5.4 Workflow 2 : Guided Search

Workflow 2 – Guided Search

Table $R \rightarrow$ **Agent LLM** $\rightarrow$ *Transformations + Candidats prioritaires* $\rightarrow$
Algo : valide $\rightarrow$ PFDs

Le Workflow 2 va plus loin : après avoir suggéré les transformations, l'agent **priorise les paires** (T_X, Y) les plus prometteuses. Seuls les candidats jugés **high** ou **medium** par le LLM sont validés.

Exemple de priorisation (Llama sur t1.csv) :

```

1 {
2   "x_transformation": "prefix(Full Name, 1)",
3   "y_column": "Gender",
4   "priority": "high",
5   "reason": "Le premier token du nom determine le sexe"
6 }
```

5.5 Workflow 3 : Agent-in-the-Loop

Workflow 3 – Agent-in-the-Loop Discovery

Agent LLM $\xrightarrow{\text{hypothèses}}$ **Algorithme** $\xrightarrow{\text{feedback}}$ **Agent LLM** $\xrightarrow{\text{raffinements}}$
Algorithme $\rightarrow \dots \rightarrow$ PFDs

Contrairement aux workflows 1 et 2, le Workflow 3 intègre l'agent dans une **boucle itérative de raffinement**. Il s'agit du workflow le plus avancé présenté dans le cours (slides 31-34).

Algorithme général ([`agentic_workflow.py` :264-401]) :

Algorithm 2 Workflow 3 – Agent-in-the-Loop

Require: DataFrame D , LLM, K , θ , max_iter

- 1: hypotheses₀ $\leftarrow$ LLM.propose_initial_hypotheses(schema(D))
- 2: accepted $\leftarrow \emptyset$
- 3: **for** $t = 0$ **to** max_iter **do**
- 4: strong, weak, rejected $\leftarrow$ validate(hypotheses _{t} , D , K , θ)
- 5: accepted $\leftarrow$ accepted $\cup$ strong
- 6: **if** weak = $\emptyset$ **and** rejected = $\emptyset$ **then**
- 7: **break** {Convergence}
- 8: **end if**
- 9: hypotheses _{$t+1$} $\leftarrow$ LLM.refine(strong, weak, rejected, K , θ)
- 10: **end for**
- 11: **return** accepted

Phase de raffinement. Après validation, l’agent reçoit un **feedback structuré** listant trois catégories de règles :

- **Règles fortes** (conf $\geq \theta$) : conservées dans l’ensemble accepté.
- **Règles faibles** (conf $< \theta$) : l’agent doit proposer une alternative plus stricte (ex. : `prefix(ZIP, 2)` avec conf=0.6 $\Rightarrow$ essayer `prefix(ZIP, 3)`).
- **Règles rejetées** (support $< K$) : l’agent peut les abandonner ou modifier la transformation.

L’agent peut aussi proposer de **nouvelles hypothèses** sur des paires (X, Y) non encore testées.

Exemple sur US_Phone_Code.csv (51 lignes, 3 colonnes `State`, `Short`, `Code`) :

- *Itération 0* : Mistral propose 7 hypothèses dont `identity(State)  $\rightarrow$  Code` et `first_token(State)  $\rightarrow$  Code`. Résultat : 6 fortes, 1 faible, 0 rejetée.
- *Itération 1* : Mistral raffine la règle faible et propose 2 nouvelles hypothèses (`numeric_prefix(Code, 1)  $\rightarrow$  Short`, etc.). Résultat : 2 fortes, 0 faible, 0 rejetée.
- Convergence atteinte. **Total : 7 PFDs acceptées, dont 5 parfaites** (conf. = 1.0), en 107 s.

5.6 Généralisation sémantique par LLM

L’étape 5 du pipeline a été étendue avec une variante **sémantique par LLM** qui regroupe les PFDs par **concept métier** ([generalization.py :91-160]).

Principe. On demande au LLM de regrouper les PFDs découvertes en clusters sémantiques (concepts métier), puis de choisir une règle représentante par cluster.

Exemple de groupes formés par Mistral sur t1.csv :

TABLE 6 – Exemples de regroupements sémantiques produits par Mistral 7B

Concept métier	Règle représentante	Membres
Le code département détermine son nom	<code>identity(Department) → Dpt Name</code>	1
Code de division → catégorie d’affectation	<code>numeric_prefix(Division, 1) → Assign. Cat</code>	4
Le prénom détermine le genre	<code>first_token(Name) → Gender</code>	2
Le nom de département détermine la division	<code>identity(Dpt Name) → Division</code>	1
Titre de poste → catégorie d’affectation	<code>identity(Position Title) → Assign. Cat</code>	1

Sur `t1.csv`, la généralisation sémantique a regroupé les 30 meilleures PFDs validées en **10 concepts métier**, contre 85 règles produites par la généralisation purement algorithmique — une réduction de $\sim 67\%$ en plus, avec une meilleure interprétabilité.

5.7 Comparaison des workflows

TABLE 7 – Comparaison structurelle des quatre approches

	Classique	W1	W2	W3
Choix des transformations	Auto	LLM	LLM	LLM (itér.)
Sélection des candidats	Exhaust.	Exhaust.	LLM	LLM (itér.)
Validation	Algo	Algo	Algo	Algo
Feedback algo → LLM	—	—	—	Oui
Généralisation	Algo	Algo	Algo	Algo
Généralisation sém. (LLM)	Dispo.	Dispo.	Dispo.	Dispo.
Appels LLM	0	1	2	$1 + 2k$

6 Résultats expérimentaux

6.1 Paramètres expérimentaux

Toutes les expériences utilisent les mêmes paramètres :

- Support minimum : $K = 5$
- Confiance minimum : $\theta = 0.85$
- Longueur maximale des préfixes : 4
- Machine : MacBook Air M2, 16 Go RAM

6.2 Résultats de l’approche classique

Le tableau 8 présente les résultats de l’approche classique sur les 14 datasets.

TABLE 8 – Résultats de l’approche classique sur les principaux datasets

Dataset	Candidats	Valides	Généralisées	Temps (s)	Conf. max
t1.csv	544	143	85	8.56	1.000
t2.csv	1 428	327	190	12.73	1.000
t3.csv	704	105	63	2.78	1.000
mechanism_refs.csv	172	20	15	4.62	1.000
research_companies.csv	148	32	24	0.46	1.000
570-1.csv	544	143	85	8.48	1.000
6397-1.csv	800	24	22	8.63	1.000

6.2.1 Exemples de PFDs découvertes

Dataset t1.csv (employés gouvernementaux, 9 101 lignes) :

TABLE 9 – Top PFDs découvertes sur t1.csv (classique)

PFD		Support	Confidence
identity(Department)	→ Department Name	9 090	1.000
identity(Department Name)	→ Department	9 090	1.000
identity(Underfilled Job Title)	→ Assignment Cat.	998	0.996
numeric_prefix(Division, 1)	→ Assignment Cat.	2 232	0.995
identity(Division)	→ Department	8 527	0.987

La PFD `identity(Department) → Department Name` est une FD classique parfaite : chaque code de département détermine de manière unique le nom du département (confiance = 1.0, support = 9 090).

Dataset t2.csv (entreprises, 3 502 lignes) :

TABLE 10 – Top PFDs découvertes sur t2.csv (classique)

PFD		Support	Confidence
first_token(FAX)	→ COUNTRY	2 239	1.000
identity(NAME)	→ COUNTRY	1 994	1.000
identity(PHONE)	→ COUNTRY	1 952	1.000
identity(ADDRESS_1)	→ COUNTRY	1 774	1.000
last_token(FAX)	→ COUNTRY	1 605	1.000

Dataset mechanism_refs.csv (références pharmaceutiques, 9 536 lignes) :

La PFD `first_token(ref_id) → ref_type` (support = 1 776, confiance = 1.0) capture le fait que le premier token d’un identifiant de référence détermine son type (PubMed, DailyMed, Wikipedia).

6.3 Résultats de l’approche agentique

6.3.1 Comparaison globale sur t1.csv

Le tableau 11 présente la comparaison des 5 approches sur le dataset **t1.csv**.

TABLE 11 – Comparaison des approches sur t1.csv (9 101 employés)

Approche	PFDs	Parfaites	Candidats	Conf. moy	Supp. moy	Temps (s)
Classique	85	2	544	0.913	7 482	8.56
W1-Mistral	23	2	72	0.931	6 747	70.75
W2-Mistral	10	0	34	0.939	5 737	51.76
W1-Llama	12	0	40	0.915	7 804	62.52
W2-Llama	21	2	74	0.934	7 145	99.51

6.3.2 Analyse de la réduction de l’espace de recherche

L’un des avantages clés de l’approche agentique est la **réduction drastique de l’espace de recherche** :

TABLE 12 – Réduction de l’espace de recherche par rapport au classique

Approche	Candidats	Réduction	PFDs trouvées
Classique	544	–	85
W1-Mistral	72	–86.8%	23
W2-Mistral	34	–93.8%	10
W1-Llama	40	–92.6%	12
W2-Llama	74	–86.4%	21

Le Workflow 2 avec Mistral réduit l’espace de recherche de **93.8%**, ne validant que 34 candidats au lieu de 544, tout en maintenant une confiance moyenne supérieure (**0.939** vs. 0.913).

6.3.3 Suggestions des agents

Mistral (Workflow 1, t1.csv) a suggéré **9 transformations**, incluant :

- `first_token(Full Name)` : « le premier mot du nom complet est le prénom »
- `prefix(Date First Hired, 7)` : « les 7 premiers chiffres de la date d’embauche déterminent l’année »
- `identity(Department)`, `identity(Division)`, etc.

Llama (Workflow 1, t1.csv) a suggéré **5 transformations**, incluant :

- `first_token(Full Name)` : « le premier mot du nom complet est le prénom, souvent unique »
- `suffix(Division, 20)` : « les 20 derniers caractères de la division indiquent la spécialisation »
- `length(Position Title)` : « la longueur du titre peut classer les employés par niveau »

On note que les deux LLMs identifient correctement `first_token(Full Name)` comme transformation pertinente, mais divergent sur les transformations secondaires — Llama propose des transformations plus originales (`suffix`, `length`) tandis que Mistral se concentre sur les `identity`.

6.3.4 Candidats prioritaires (Workflow 2)

Mistral a priorisé 4 candidats :

1. `prefix(Department Name, 10) → Department` — priorité high
2. `prefix(Division, 35) → Division` — priorité medium
3. `prefix(Underfilled Job Title, 10) → Job Title` — priorité medium
4. `first_token(Position Title) → Job Role` — priorité low

Llama a priorisé 4 candidats :

1. `prefix(Full Name, 1) → Gender` — priorité high
2. `first_token(Department Name) → Department` — priorité high
3. `prefix(Date First Hired, 1) → Month of Hire` — priorité medium
4. `numeric_prefix(Division, 1) → Division Type` — priorité medium

Llama propose des candidats sémantiquement plus riches (`prefix(Full Name, 1) → Gender` est une véritable PFD originale), tandis que Mistral reste plus conservateur.

6.4 Comparaison Mistral vs. Llama

TABLE 13 – Comparaison des deux LLMs

Critère	Mistral 7B	Llama 3.1 8B
Nombre de suggestions (W1)	9	5
Conf. moyenne (W1)	0.931	0.915
Candidats explorés (W2)	34	74
Conf. moyenne (W2)	0.939	0.934
Temps moyen par workflow	~61s	~81s
Style de suggestions	Conservateur	Créatif
Réponse JSON	Plus fiable	Parfois verbeux

Mistral produit des résultats plus **précis** (confiance moyenne supérieure) avec un espace de recherche plus réduit (34 candidats en W2). **Llama** propose des transformations plus **diversifiées et originales** (suffix, length), mais explore un espace plus large (74 candidats en W2).

6.5 Résultats du Workflow 3 (Agent-in-the-Loop)

Nous avons validé le Workflow 3 sur le dataset `US_Phone_Code.csv` (51 lignes, 3 colonnes : `State`, `Short`, `Code`). Le tableau 14 détaille le déroulement des itérations.

TABLE 14 – Déroulement des itérations du Workflow 3 (Mistral sur US_Phone_Code)

Iter	Hyp.	Fortes	Faibles	Rejet.	Hypothèses proposées
0	7	6	1	0	<code>identity(State)→Code</code> , <code>prefix(Short, 2)→Code</code> , <code>length(State)→Code</code> , etc.
1	4	2	0	0	2 raffinements + 2 nouvelles hypothèses (<code>numeric_prefix(Code, 3)→Short</code> , etc.)

Bilan final :

- 11 hypothèses testées au total (au lieu de ~ 24 candidats explorés par l’approche classique).
- **7 PFDs acceptées**, dont **5 parfaites** (confidence = 1.0).
- **Convergence** atteinte dès l’itération 1 (aucune règle faible restante).
- Confiance moyenne : 0.978. Temps d’exécution : 107 s.

Observation clé sur W3

La **capacité d’auto-correction** du Workflow 3 est observée : à l’itération 0, Mistral propose `length(State)→Code` qui obtient une faible confiance. À l’itération 1, suite au feedback, l’agent **abandonne cette règle** et propose à la place `numeric_prefix(Code, 3)→Short` qui s’avère parfaite (conf = 1.0). Cette boucle de raffinement est le cœur de l’approche agentique autonome.

Résultats sur t1.csv (9101 lignes, 9 colonnes). Sur ce dataset plus large, le Workflow 3 a convergé en 3 itérations.

TABLE 15 – Déroulement du Workflow 3 (Mistral sur t1.csv)

Iter.	Hyp. testées	Fortes	Faibles	Rejetées
0	7	5	2	0
1	5 (3 raffin. + 2 nouv.)	2	1	0
2	5 (3 raffin. + 2 nouv.)	3	1	0
Total	17	—	—	—

Bilan sur t1.csv :

- **17 hypothèses testées** (au lieu de 544 candidats par l’approche classique : réduction de **96.9%**).
- **8 PFDs acceptées** après généralisation, dont **3 parfaites** (confidence = 1.0) :
 - `identity(Full Name) → Gender`
 - `identity(Department) → Department Name`
 - `identity(Department Name) → Department`
- Confiance moyenne : **0.95**. Temps d’exécution : 361 s.

Notamment, W3-Mistral découvre `identity(Full Name) → Gender` qui n'apparaît pas dans le top des résultats du classique (qui se concentre sur des règles de plus haut support). Cela illustre l'apport sémantique du LLM pour la découverte ciblée.

TABLE 16 – Comparaison W3 vs. Classique vs. W2 sur t1.csv

Approche	Candidats	PFDs	Parfaites	Temps (s)
Classique	544	85	2	8.5
W2-Mistral (après fix)	34	19	2	51.8
W3-Mistral (3 iter.)	17	8	3	361.4

W3 trouve **plus de PFDs parfaites** (3 contre 2) avec **deux fois moins de candidats** que W2, au prix d'un temps d'exécution multiplié par 7 (les 3 itérations LLM dominant le coût).

6.6 Analyse de sensibilité aux seuils K et θ

Nous avons étudié l'effet des seuils sur le nombre de PFDs découvertes par l'approche classique sur `t1.csv`. Le tableau 17 présente les résultats pour 12 configurations ($K \in \{5, 10, 20\}$, $\theta \in \{0.85, 0.90, 0.95, 1.00\}$).

TABLE 17 – Sensibilité de la découverte aux seuils sur t1.csv (approche classique)

K	θ	Nb PFDs	PFDs parfaites	Conf. moy.	Temps (s)
5	0.85	85	2	0.913	11.67
5	0.90	54	2	0.933	11.36
5	0.95	16	2	0.982	11.59
5	1.00	3	3	1.000	11.92
10	0.85	85	3	0.913	8.70
10	0.90	53	3	0.934	8.41
10	0.95	17	3	0.981	8.39
10	1.00	4	4	1.000	8.88
20	0.85	85	3	0.914	10.73
20	0.90	55	3	0.935	13.87
20	0.95	19	3	0.979	10.69
20	1.00	4	4	1.000	9.75

Enseignements :

1. **Effet majeur de θ** : passer de $\theta = 0.85$ à $\theta = 0.95$ divise par 5 le nombre de PFDs ($85 \rightarrow 16-19$). À $\theta = 1.0$, on ne garde que les FDs strictes (3-4 règles).
2. **Effet mineur de K** : passer de $K = 5$ à $K = 20$ ne change que très peu le nombre de PFDs (le filtrage par confiance domine).
3. **Confiance moyenne croissante avec θ** : 0.913 à $\theta = 0.85$, 0.982 à $\theta = 0.95$, 1.0 à $\theta = 1.0$.
4. **Temps quasi-constant** : la complexité algorithmique est dominée par la phase de validation, indépendante des seuils.

Le choix $\theta = 0.85$ retenu dans nos expériences principales est un **compromis raisonnable** : il maximise la découverte (85 PFDs) tout en gardant une confiance moyenne acceptable (> 0.91). Pour une utilisation en audit qualité (faible tolérance au bruit), $\theta = 0.95$ serait plus adapté.

6.7 Généralisation sémantique par LLM vs. algorithmique

Nous avons comparé les deux stratégies de généralisation sur les 30 meilleures PFDs validées sur `t1.csv`.

TABLE 18 – Comparaison des stratégies de généralisation sur `t1.csv`

Stratégie	PFDs sortie	Réduction	Interprétabilité
Algorithmique (143 PFDs en entrée)	85	41%	Syntaxique
LLM-Mistral (30 meilleures)	27	10%	Concepts métier

Avantage qualitatif majeur de la généralisation LLM : chaque groupe est nommé par un concept en langage naturel (cf. tableau 6). Par exemple, le groupe « Le code de division détermine la catégorie d’affectation » fusionne 4 PFDs (`numeric_prefix(Division, k)` pour $k \in \{1, 2, 3, 4\}$) que la généralisation algorithmique classerait séparément.

Limites observées :

- Le LLM ne traite que 30 PFDs en entrée (limite de contexte de Mistral 7B). Au-delà, on observe des timeouts.
- Le LLM peut occasionnellement générer des concepts métier redondants si le prompt n’est pas suffisamment contraint.

6.8 Analyse d’erreur : pourquoi Mistral W2 manquait des PFDs parfaites

Dans la version initiale du projet, W2-Mistral avait manqué les 2 PFDs parfaites (confiance = 1.0) que l’approche classique avait découvertes sur `t1.csv`. Le script `error_analysis.py` compare automatiquement les PFDs parfaites des deux approches et classe chaque PFD manquée en trois catégories de cause :

1. **Transformation jamais suggérée** : l’agent n’a pas inclus la transformation T_X dans sa liste de transformations pertinentes (phase 1).
2. **Transformation OK mais paire non priorisée** : T_X est dans la liste, mais la paire (T_X, Y) n’a pas été sélectionnée par l’agent (phase 2).
3. **Paire priorisée mais filtrée par seuil** : la paire a été validée mais a échoué aux seuils K ou θ .

Résultats du diagnostic initial (avant correction) :

TABLE 19 – Diagnostic des PFDs parfaites manquées par W2-Mistral sur t1.csv

PFD parfaite manquée		Cause	Diagnostic
identity(Department) Department Name	→	Non priorisée	Mistral n'avait proposé que <code>prefix(Department Name, 10)</code> ; la version <code>identity</code> était absente des candidats prioritaires.
identity(Dpt Name) Department	→	Non priorisée	Même problème : pas d' <code>identity</code> proposée pour ce couple.

Cause racine identifiée : le prompt original `CANDIDATE_PRIORITIZATION_PROMPT` insistait sur le filtrage des candidats « triviaux ou absurdes ». Le LLM interprétait `identity(col)` (qui correspond aux FDs classiques) comme « trivial » et l'écartait systématiquement. Or, c'est précisément cette transformation qui capture les FDs classiques parfaites.

Correction apportée ([llm_agent.py :60-65]) : ajout d'une instruction explicite dans le prompt :

« *IMPORTANT : pour les colonnes catégoriques avec peu de valeurs uniques (codes de département, ID, abbreviations), n'oublie PAS de proposer la transformation `identity(col)` qui correspond aux Dépendances Fonctionnelles classiques (souvent *confidence = 1.0*). »*

Résultats après correction :

TABLE 20 – Effet de la correction du prompt sur W2-Mistral

	Avant	Après
PFDs parfaites trouvées par classique	2	2
PFDs parfaites trouvées par W2-Mistral	0	2
PFDs parfaites manquées	2	0

Cette analyse illustre une leçon importante : **la qualité d'un workflow agentique dépend autant du contenu du prompt que du choix du LLM**. Une instruction explicite peut combler un biais systématique du modèle.

7 Discussion

7.1 Avantages de l'approche agentique

1. **Réduction massive de l'espace de recherche** : les workflows agentiques explorent 87–94% moins de candidats que l'approche classique, grâce au filtrage sémantique du LLM.
2. **Confidence moyenne supérieure** : en se concentrant sur des candidats sémantiquement pertinents, les approches agentiques obtiennent des PFDs de meilleure qualité (0.931–0.939 vs. 0.913 en classique).
3. **Interprétabilité** : chaque suggestion du LLM est accompagnée d'une **justification en langage naturel**, facilitant l'interprétation des résultats.

4. **Filtrage des dépendances triviales** : le LLM évite les PFDs sans sens sémantique (par exemple, `length(col) → col2` qui tiennent par coïncidence).

7.2 Limites et compromis

1. **Temps d'exécution** : l'inférence LLM ajoute 50 à 100 secondes de surcoût (8.5s en classique vs. 51–100s en agentique). Ce surcoût est fixe et ne dépend pas de la taille du dataset.
2. **Couverture** : l'approche classique découvre **85 PFDs** contre 10–23 en agentique. Certaines PFDs valides mais non intuitives sont manquées par le LLM.
3. **Déterminisme** : même avec une température très basse (0.1), les réponses LLM peuvent varier légèrement entre exécutions.
4. **Parsing JSON** : les LLMs ne respectent pas toujours le format JSON demandé, nécessitant un parsing robuste avec fallback (regex `\{[\s\S]*\}`).
5. **PFDs parfaites initialement manquées** : dans la version initiale du projet, W2-Mistral ne trouvait aucune PFD parfaite (confiance = 1.0) alors que le classique en trouvait 2. Notre analyse d'erreur (section 6.8) a identifié la cause (biais du LLM envers `identity(col)`) et la correction du prompt a résolu le problème : W2-Mistral retrouve maintenant les 2 PFDs parfaites. Cet exemple illustre l'importance du **prompt engineering** dans les approches agentiques.

7.3 Analyse du compromis qualité/exhaustivité

Le tableau 21 résume le compromis fondamental entre les approches.

TABLE 21 – Compromis qualité vs. exhaustivité

	Exhaust.	Précision	Vitesse	Interpr.	Auto-correc.
Classique	***	**	***	*	—
Workflow 1	**	***	*	***	—
Workflow 2	*	***	**	***	—
Workflow 3	**	***	*	***	***
Gén. LLM (après)	N/A	***	**	***	—

L'approche classique excelle en **exhaustivité** et **vitesse**, tandis que les approches agentiques excellent en **précision** et **interprétabilité**. Le choix dépend du cas d'usage : l'exploration exhaustive est préférable pour l'audit de qualité de données, tandis que l'approche agentique convient mieux à la découverte ciblée de règles métier.

7.4 Reproductibilité

Tous les résultats sont reproductibles :

- Les LLMs sont exécutés **localement** via Ollama (pas de dépendance à des APIs cloud).
- Les paramètres ($K = 5$, $\theta = 0.85$) sont fixés dans les scripts.
- Les résultats sont sauvegardés en JSON avec horodatage.

- Le code est publié sur GitHub : <https://github.com/omarsoliman02/PFD-Discovery>

8 Conclusion

Ce projet a permis d’implémenter et de comparer plusieurs paradigmes pour la découverte de PFDs :

1. Un **pipeline classique** en 5 étapes, explorant systématiquement toutes les combinaisons de transformations et de colonnes. Il découvre un grand nombre de PFDs (85 sur t1.csv) en un temps très court ($\sim 8s$).
2. **Trois workflows agentiques** utilisant des LLMs locaux (Mistral 7B et Llama 3.1 8B) :
 - **W1 (Feature-Enriched)** : réduction de l’espace de recherche de $\sim 87\%$.
 - **W2 (Guided Search)** : réduction de $\sim 94\%$.
 - **W3 (Agent-in-the-Loop)** : boucle itérative où l’algorithme renvoie un feedback au LLM, qui raffine ses hypothèses. Démonstre une réelle capacité d’auto-correction (cas observé : abandon de `length(State)→Code` après feedback, remplacement par `numeric_prefix(Code, 3)→Short`).
3. Une **généralisation sémantique par LLM** qui regroupe les PFDs en concepts métier interprétables (10 concepts sur t1.csv au lieu de 85 règles syntaxiques).
4. Une **analyse de sensibilité** aux seuils K et θ , montrant que θ est le facteur dominant (85 PFDs à $\theta = 0.85$, 3 PFDs à $\theta = 1.0$).
5. Une **analyse d’erreur détaillée** ayant permis d’identifier et corriger un biais de Mistral envers la transformation `identity(col)`.

Contributions principales :

- Implémentation complète en Python avec 17 modules réutilisables et bien découplés.
- Implémentation des **trois workflows agentiques** présentés dans le cours (au lieu d’un seul minimum requis).
- Comparaison systématique sur 14 datasets réels couvrant 3 domaines.
- Comparaison de 2 LLMs (Mistral vs. Llama) mettant en évidence des profils différents : Mistral est plus précis et compact, Llama est plus créatif et diversifié.
- Démonstration du compromis fondamental entre exhaustivité (classique) et précision sémantique (agentique).
- **Diagnostic d’erreur reproductible** : identification d’un biais systématique du LLM et correction par modification du prompt.

Perspectives :

- Tester des LLMs plus puissants (Mistral 8x7B, Llama 70B, GPT-4) pour améliorer la qualité des hypothèses initiales du W3.
- Explorer des métriques de qualité supplémentaires (interestingness, novelty) pour évaluer la pertinence sémantique.
- Étendre le système à la découverte de PFDs **multi-colonnes** ($X_1, X_2 \rightarrow Y$).
- Combiner W3 et généralisation sémantique LLM en un pipeline 100% agentique de bout en bout.

- Optimiser le coût en temps du W3 (cache des hypothèses, batch validation) pour le rendre compétitif avec W1/W2.

A Annexe : Guide d'utilisation

A.1 Installation

```

1 # Cloner le repository
2 git clone https://github.com/omarsoliman02/PFD-Discovery.git
3 cd PFD-Discovery
4
5 # Installer les dependances Python
6 pip install -r requirements.txt
7
8 # Installer Ollama (modeles locaux)
9 brew install ollama
10 brew services start ollama
11 ollama pull mistral          # Mistral 7B (~4.4 Go)
12 ollama pull llama3.1        # Llama 3.1 8B (~4.9 Go)

```

Listing 1 – Installation du projet

A.2 Exécution

```

1 # Approche classique sur tous les datasets
2 python3 experiments/run_classical.py
3
4 # Approche agentique (Mistral + Llama)
5 python3 experiments/run_agentic.py
6
7 # Comparaison sur un dataset spécifique
8 python3 experiments/compare_results.py --dataset t1
9
10 # Seulement Mistral, Workflow 1
11 python3 experiments/run_agentic.py --llm mistral --workflow 1

```

Listing 2 – Commandes d'exécution

B Annexe : Structure du code

```

1 PFD-Discovery/
2 |-- data/
3 |   |-- CHE/                (5 datasets chimie/biologie)
4 |   |-- DGOV/              (5 datasets gouvernance)
5 |   +-- pfd_validation/    (4 datasets validation)
6 |-- src/
7 |   |-- data_loader.py      Chargement CSV
8 |   |-- pattern_extraction.py 7 transformations
9 |   |-- pattern_grouping.py  Groupement par pattern
10 |   |-- candidate_generation.py Candidats X -> Y
11 |   |-- validation.py       Support + confidence
12 |   |-- generalization.py    Fusion de patterns
13 |   |-- classical_pipeline.py Pipeline 5 etapes
14 |   |-- llm_agent.py         Integration Ollama
15 |   |-- agentic_workflow.py Workflow 1 + 2
16 |   +-- evaluation.py        Metriques

```



```
17 | -- experiments/  
18 | | -- run_classical.py      Script classique  
19 | | -- run_agentic.py       Script agentique  
20 | |-- compare_results.py    Comparaison  
21 | -- results/               Resultats JSON  
22 |-- requirements.txt
```

Listing 3 – Arborescence du projet